

第 12 章 查询处理

查询处理流程

执行计划

算子实现

庖丁解牛

庖丁为文惠君解牛，手之所触，肩之所倚，足之所履，膝之所踦，砉然向然，奏刀騞然，莫不中音。合于《桑林》之舞，乃中《经首》之会。

文惠君曰：“嘻，善哉！技盖至此乎？”

庖丁释刀对曰：“臣之所好者，道也，进乎技矣。始臣之解牛之时，所见无非牛者。三年之后，未尝见全牛也。方今之时，臣以神遇而不以目视，官知止而神欲行。依乎天理，批大郤，导大窾，因其固然，技经肯綮之未尝，而况大軱乎！良庖岁更刀，割也；族庖月更刀，折也。今臣之刀十九年矣，所解数千牛矣，而刀刃若新发于硎。彼节者有间，而刀刃者无厚；以无厚入有间，恢恢乎其于游刃必有余地矣，是以十九年而刀刃若新发于硎。虽然，每至于族，吾见其难为，怵然为戒，视为止，行为迟。动刀甚微，謦然已解，如土委地。提刀而立，为之四顾，为之踌躇满志，善刀而藏之。”

文惠君曰：“善哉！吾闻庖丁之言，得养生焉。”

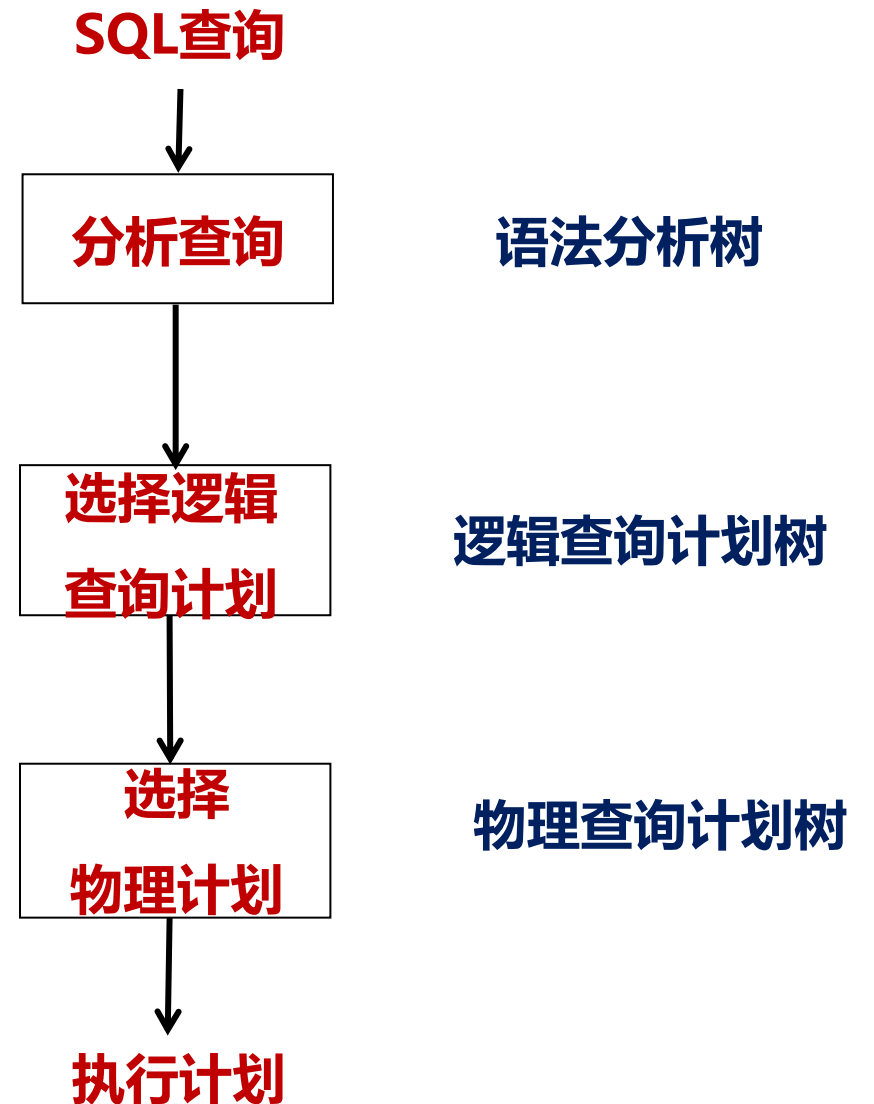
查询处理步骤

输入：SQL语句

输出：满足查询条件的数据

查询处理是指从数据库中提取数据的一系列活动，主要包括：

- 将用高层数据库语言表示的查询语句翻译为能在文件系统这一物理层次上实现的表达式
- 为优化查询而进行各种转换
- 查询的实际执行



SQL漫游记

阶段	庖丁解牛的过程	SQL执行过程
① 接牛	庖丁收到一头牛	客户端发送SQL到数据库
② 观牛	庖丁绕牛观察结构	解析器检查语法，生成语法树
③ 识牛	庖丁识别骨骼纹理	语义分析，检查表/字段是否存在
④ 定刀	庖丁选定下刀路线	优化器生成执行计划
⑤ 运刀	庖丁开始动手解牛	执行器开始逐算子执行
⑥ 批郤	刀刃沿着关节缝隙切	索引扫描/分区裁剪
⑦ 导窾	引刀进入骨节空隙	条件过滤/谓词下推
⑧ 收刀	庖丁轻松分离筋骨	数据拼接/聚合计算
⑨ 献肉	庖丁将解好的肉呈现	结果集返回给客户端

查询优化

- **查询优化**：对一个给定的查询，从多种可能执行方式中选择出最有效率的作为查询执行计划
- **基于代价的方法**(cost-based)：生成所有可能的候选计划，通过统计信息和存取路径确定每一计划中各个操作的代价和综合代价，最后选择代价最小的一个；优化代价较高
- **启发式方法**(rule-based)：主要是代数优化，依据关系代数的等价变换做一些逻辑变换；假定因素太多，优化的代价虽小，但优化的准确程度较差

关系代数表达式的优化

关系代数表达式的等价规则

各种结合律、分配律、交换律

- **谓词下推** (Predicate Pushdown) : 把过滤条件“推”到数据源附近, 减少早读数据
- **列剪枝** (Column Pruning) : 只保留查询需要的列, 减少I/O
- **子查询消除**: 将相关子查询改写为连接或聚合
- **投影合并/消除**: 合并冗余的投影操作

基于规则的优化示例

```
create table test (col1 unique, col2)
```

```
select col1, sum(col2)  
group by col1
```

系统并不执行分组操作

查询代价估算

访问方法	估计代价（逻辑读的数量）
表扫描	表所占用的全部数据页的数量
聚簇索引	索引的高度 + 要扫描的页的数量（要扫描的页的数量 = 满足条件的行的数量/每个数据页所包含的行的数量）
堆上的非聚簇索引	索引的高度 + 叶结点所指的页面数 + 满足条件的行的数量（可能所有的行均不在同一页中，所以要得到一行就必须读一次逻辑读）。由于相同的数据页经常要读取多次（从缓冲区中读取），所以这样逻辑读的数量将比表的页面数要多
聚簇索引上的非聚簇索引	索引的高度 + 叶结点所指的页面数 + （满足条件的行的数量 × 搜索一个聚簇索引码的代价）
非聚簇覆盖索引	索引的高度 + 索引的叶结点所占用的索引页（满足条件的行数/每个叶页面包含的行数）

统计信息

关系 R 的统计信息

- $T(R)$: R中的元组数
- $S(R)$: R中每个元组的字节数
- $B(R)$: R中所有元组所占据的块数
- $V(R, A)$: R的属性A上的唯一值的个数
- $f(R)$: 每个块上的R中元组的最大数目

索引 I 的统计信息

- $HT(i)$: 索引的层数
- $LB(i)$: 索引叶结点的块数

统计信息的生成方式

- 抽样
- 查询反馈(query feedback):
根据查询结果修正统计数据
- 高维统计信息: AI for DB,
学习型索引

查询处理流程

执行计划

算子实现

各种访问方法

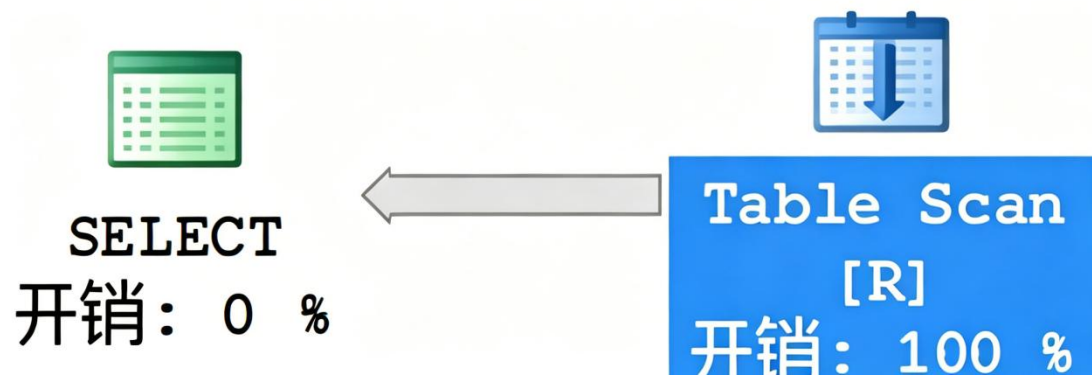
R(A, B, C, P)

查询 1: (与该批有关的) 查询开销: 100%

`select * from R`

表扫描

`select * from R`



各种访问方法

create clustered index cidx1 on R(P)

无序聚簇索引扫描

select * from R

查询 1: (与该批有关的) 查询开销: 100%

select * from R

按聚簇索引叶节点的
磁盘页物理顺序访问



各种访问方法

有序聚簇索引扫描

`select * from R order by P`

(索引碎片)

查询 1: (与该批有关的) 查询开销: 100%

`select * from R`

按聚簇索引叶节点的
磁盘页链接顺序访问



SELECT
开销: 0 %



Clustered Index Scan (Clustered)
[R].[cidx1]
开销: 100 %

各种访问方法

create index idx_A on R(A)

无序覆盖非聚簇索引扫描

select A from R

查询 1: (与该批有关的) 查询开销: 100%

select A from R



有序覆盖非聚簇索引扫描

select A from R order by A

查询 1: (与该批有关的) 查询开销: 100%

select A from R



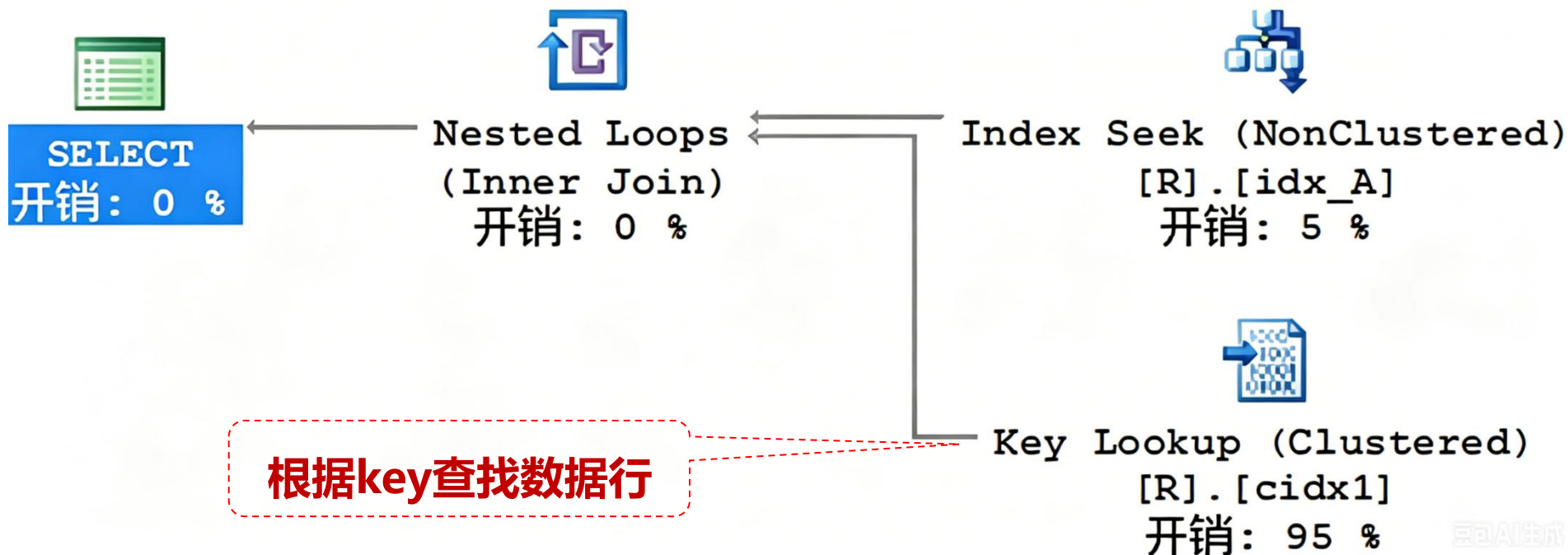
各种访问方法

非聚簇索引查找 + 有序局部扫描 + Lookups

select * from R where A between 10 and 20

查询 1: (与该批有关的) 查询开销: 100%

SELECT * FROM [R] WHERE [A]>=@1 AND [A]<=@2



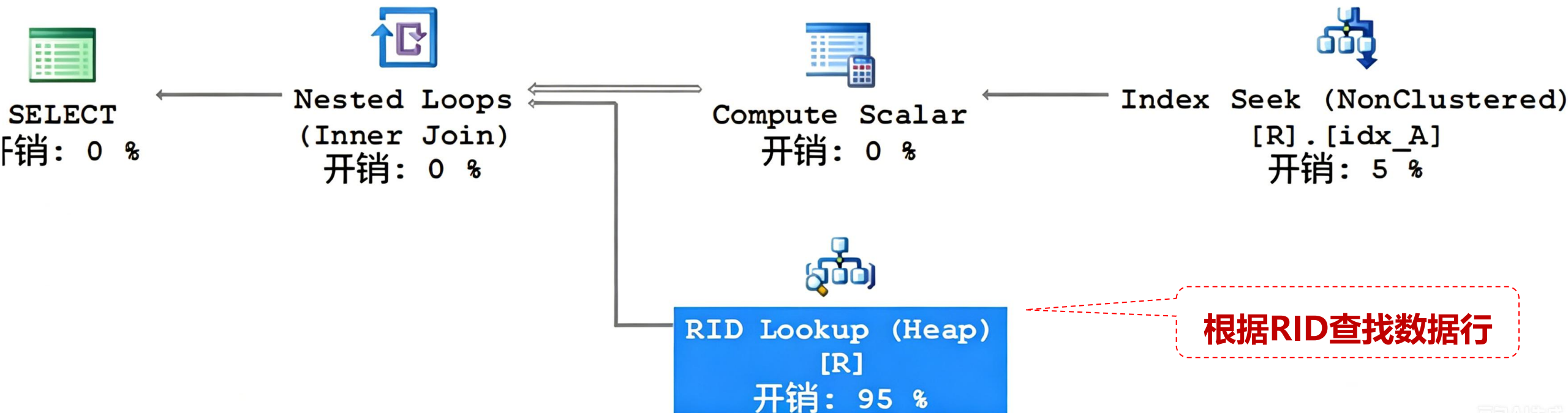
各种访问方法

非聚簇索引查找 + 有序局部扫描 + Lookups

select * from R where A between 10 and 20

查询 1: (与该批有关的) 查询开销: 100%

ELECT * FROM [R] WHERE [A]>=@1 AND [A]<=@2



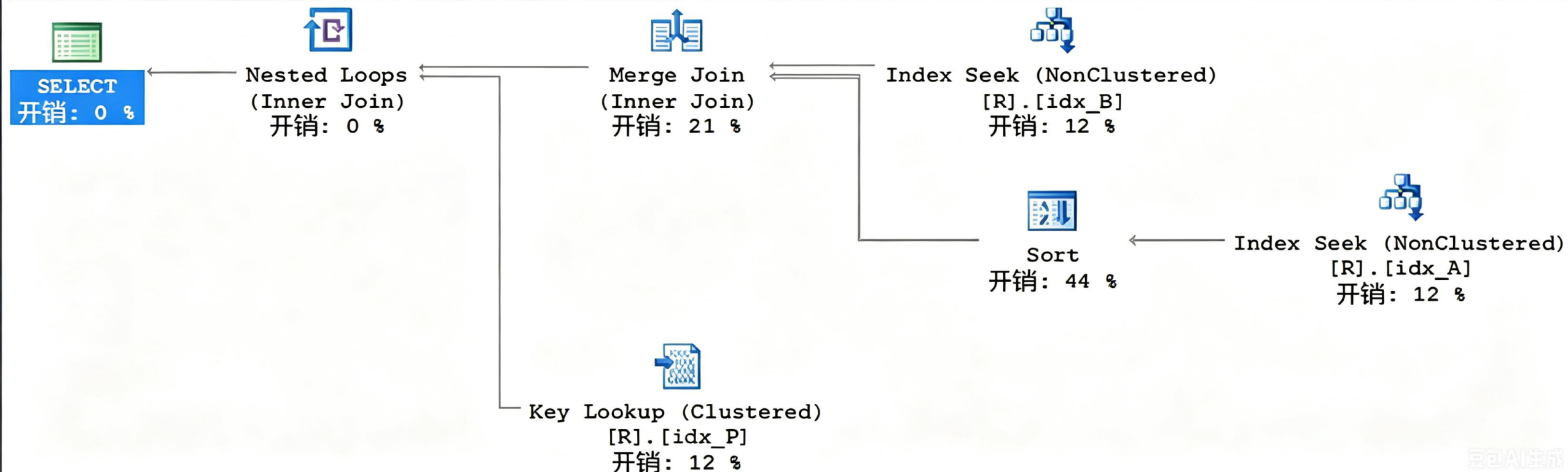
各种访问方法

索引交集

create index idx_A on R(A), create index idx_B on R(B)
select A, B, C from R where A < 100 and B = 'C0000005794'

查询 1: (与该批有关的) 查询开销: 100%

SELECT [A],[B],[C] FROM [R] WHERE [A]<@1 AND [B]=@2



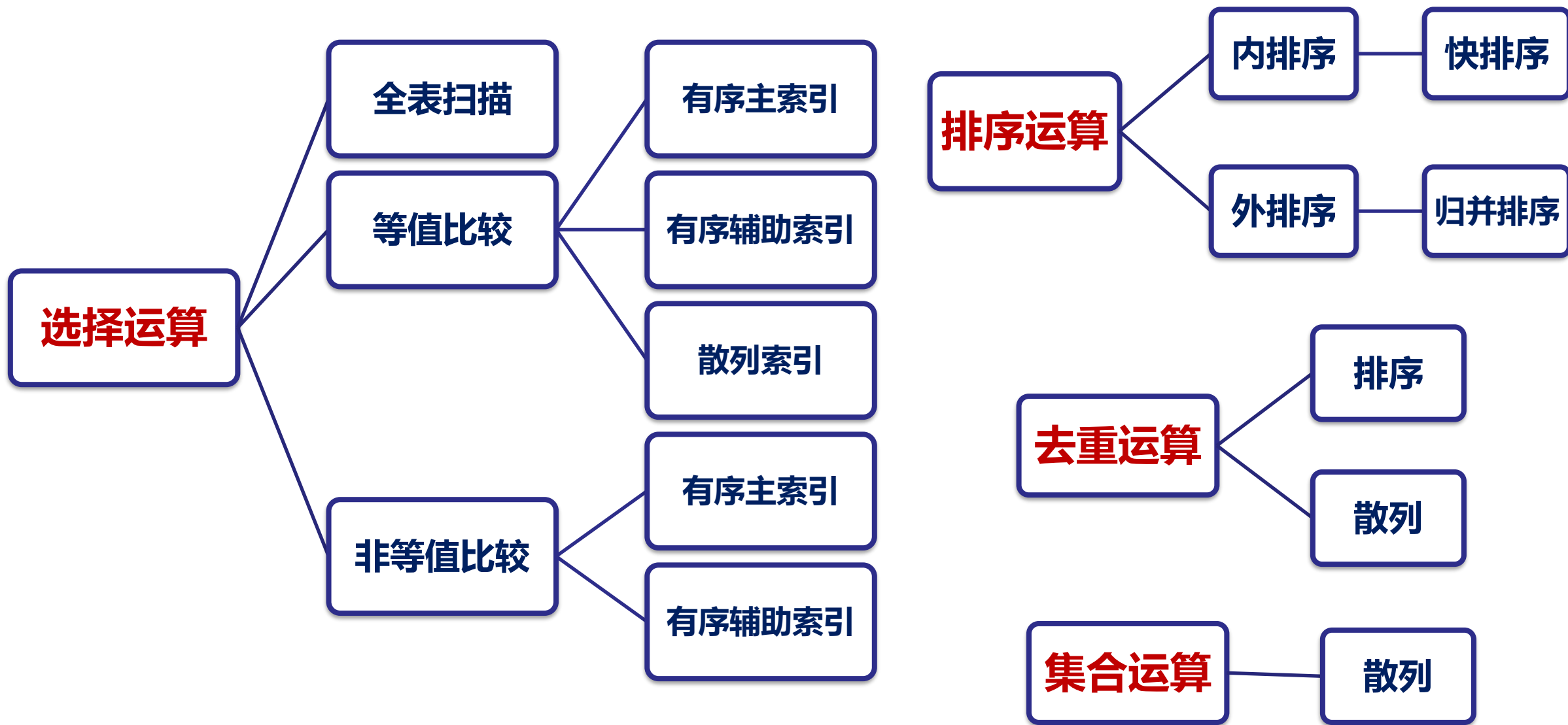


查询处理流程

执行计划

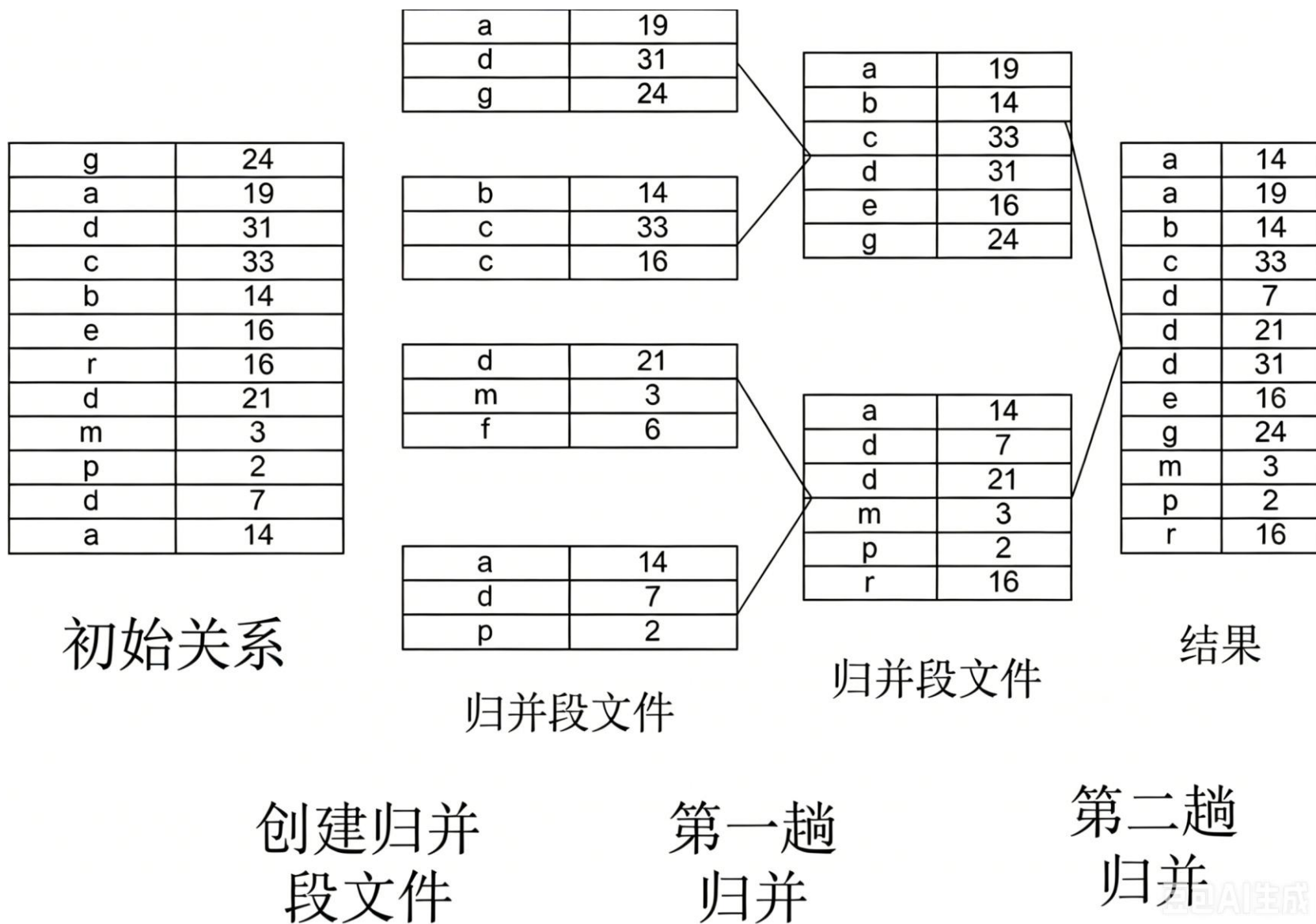
算子实现

实现各种逻辑运算的物理算子概览

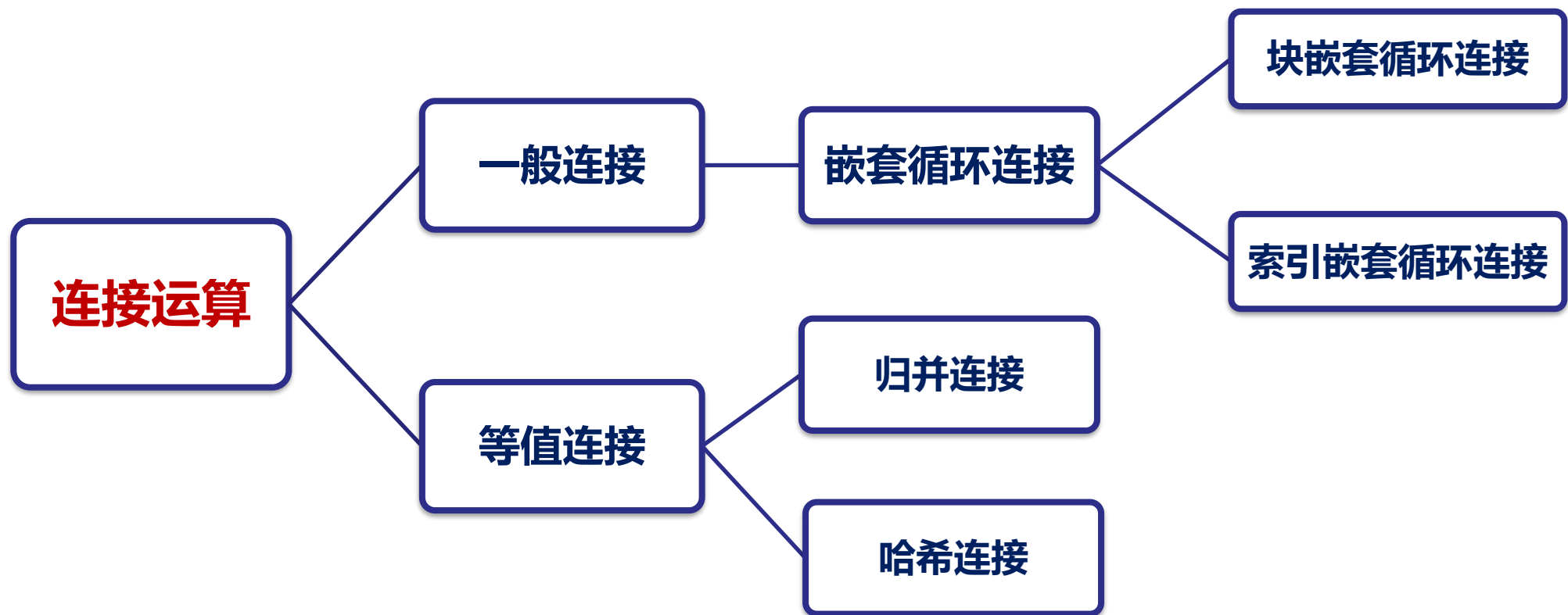


多路归并排序

- 对于可放进内存的关系，使用如快排序之类的技术
- 对不能放进内存的关系，使用外排序
- 外排序：创建有序段 + N路归并
- 所有的输入数据最初分成许多有序的归并段文件，然后不断归并成许多更大的归并段文件，直到剩下一个文件为止



实现各种逻辑运算的物理算子概览



嵌套循环连接

计算 θ 连接 $r \bowtie_{\theta} s$

for each tuple t_r in r do begin

for each tuple t_s in s do begin

测试元组对 (t_r, t_s) 是否满足连接条件 θ

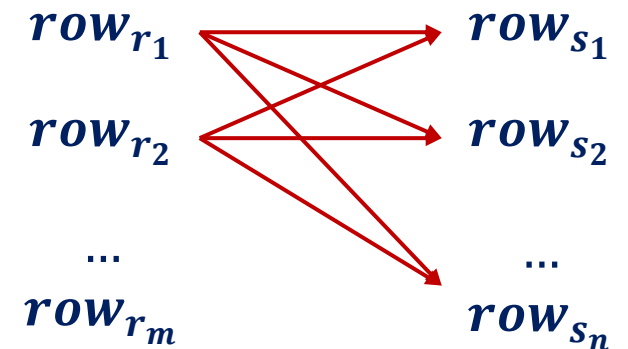
如果是, 将 $t_r \cdot t_s$ 加入到结果中

end

end

r 称为连接的外关系, s 称为内关系

不需要索引, 可用于任何连接条件



块嵌套循环连接

for each block B_r of r do begin

for each block B_s of s do begin

for each tuple t_r in B_r do begin

for each tuple t_s in B_s do begin

检查 (t_r, t_s) 是否满足连接条件

如果是, 将 $t_r \cdot t_s$ 加入结果

end

end

end

end

对于块嵌套循环连接来说, 较小的表作为内表还是外表效率更高?

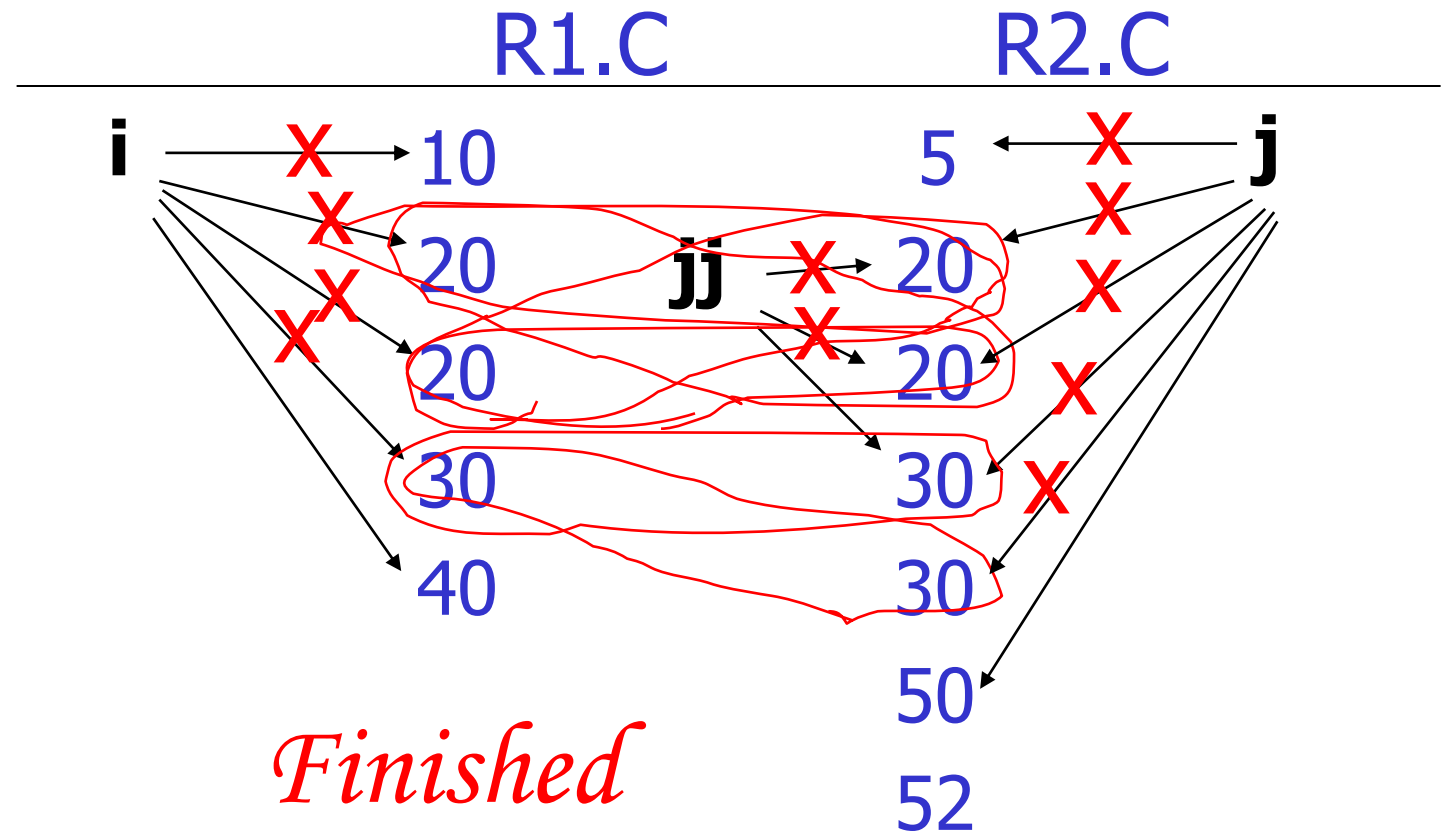
索引嵌套循环连接

- 如果满足下列条件，索引查找可以代替文件扫描
 - 是等值连接或自然连接
 - 在内关系的连接属性上存在索引
- 对外关系 r 的每个元组 tr ，利用索引查找内关系 s 的与 tr 满足连接条件的元组

对于索引嵌套循环连接来说，带索引的表作为内表还是外表效率更高？

归并连接

1. 在当前R和S块查找连接属性的最小值
2. 如果这个值在另一个关系的前部没有出现，那么就删除具有这个值的元组
3. 否则，找出两个关系中的这个值的所有元组，如果需要，从排序的R或者S中读取块，直到不再有其他的元组
4. 进行连接，输出和这个值相关的所有得到的结果元组
5. 如果一个关系在内存中已经没有要考虑的元组了，那么就重新装载为这个关系而设定的缓冲区



哈希连接

生成输入

hash: mod 4

探测输入

- 将两个关系按连接属性值划分成有相同哈希函数值的元组集合
- 关系r在一个哈希划分中的元组只需要与关系s在对应的划分中的元组相比较
- 在r和s的每一对划分中进行索引嵌套循环连接（哈希索引）

对于哈希连接来说，较小的表作为生成表还是探测表？

R1

2
4
3
5
8
9

Buckets

4, 8	4, 8, 12
5, 9	5, 13
2	14
3	3, 11

R2

5
4
12
3
13
8
11
14

各种连接策略比较

对比维度	朴素嵌套循环连接	索引嵌套循环连接	合并连接	哈希连接
支持连接条件	等值、非等值 ($> < \geq \leq \neq$)	等值、非等值	仅等值连接	仅等值连接
最优数据特征	两个关系数据量都极小	一端数据极少 (<10 行)、另一端大表连接列建有索引	两张大数据集，连接列已有序 / 索引有序	两张超大无序无索引数据，两表规模差距大
核心原理	外表逐元组，内表全表逐元组笛卡尔积比对	小表每条元组，通过大表索引快速查找匹配行	两路有序数据归并遍历，顺序匹配等值键	小表构建哈希表，大表逐行探测哈希匹配
最优适用场景	双表数据量都非常少（几行~几十行）	一端不足 10 行，另一端大表且连接列带索引	两大表连接列有序、等值连接	超大无序无索引等值连接，两表体量悬殊
额外开销	无排序、无哈希开销，纯全量遍历	无额外排序 / 哈希开销，依赖索引	数据无序时必须额外排序	数据过大内存不够，产生磁盘落地开销
劣势场景	任意一张表数据量大，性能爆炸恶化	驱动表数据量大、被连接列无索引	小数据量浪费排序成本、不能非等值	无法非等值连接、极小数据构建哈希浪费资源

多表连接如何确定连接顺序？

N个表可能的连接种类有N!个

- **对少于或等于4个的表连接采用验证全部组合的方法，对于超过4个的表连接，以4个表为一组进行验证**
- **ABCDEF6个表，先求出其任意4个表组合中代价最小的，不妨设为BCDF，则表B作为最外层表，剩余5个表ACDEF，再求出其任意4个表组合中代价最小的，不妨设为CDEF，则表C作为次外层表，最后按通常方法确定剩余表的最佳连接顺序，假设为DEFA，则最后表连接顺序为BCDEFA**

算子执行方式

物化执行(Materialization)

- 生成输入为关系或已计算结果的表达式的结果，在磁盘上物化(存储)
- 从最底层开始每次计算一个运算，将中间结果物化成临时关系，再进行下一层运算
- 物化执行总是可用的
- 将结果写到磁盘并读回来可能代价很高

流水线化(Pipelining)

- 一个操作在执行过程中就将部分输出元组送到父运算
- 流水线执行：同时计算多个运算，将一个运算的结果传到下一个运算
- 比物化执行代价要低，不需要将临时关系存储到磁盘
- 有时流水线不可用 – 如排序和散列连接
- 为了使流水线有效，使用能在接受输入元组的同时生成输出元组的执行算法（无阻塞算子）

算子封装形式：迭代器

选择

open()

open: 打开一个输入

getnext()

getnext: 一次一个项的读取每一个项，如果遇上一个项符合条件的话，就将这个项传递给下一个处理迭代器

close()

Close: 关闭输入